

DATABASE_SCHEMA

Helix Cluster OS — Database Schema Reference

PostgreSQL 16+ is the authoritative relational store for the Helix Cluster control plane: the node registry, GPU inventory, sessions and their window/pane tree, resource reservations, the scheduling queue, health snapshots, LLM advisories, the build subsystem, users, the migration-history ledger, network policy, cluster config, and an append-only audit log.

This document is grounded in the real SQL under `migrations/postgresql/`. Every table, column, constraint, index, trigger, and function below comes from those files. As of HXC-1639 there is a single canonical schema source (the `golang-migrate` chain); the consolidated `0001_primary_schema.sql` is a generated artifact kept in lock-step by a drift guard — see §1.

1. One schema source — read this first (HXC-1639 resolved)

Historically there were **two distinct, independently-applied** schema definitions in `migrations/postgresql/` and they were *not* byte-identical (different table names, missing tables, missing CHECK constraints — the old divergence table is preserved at the bottom of §1c for the record). **As of HXC-1639 that divergence is RESOLVED: there is now a single canonical source, and divergence is mechanically impossible to re-introduce silently.**

1a. The `golang-migrate` chain (001–015) is the single canonical source

The numbered pairs `001_*.up.sql ... 015_*.up.sql` (with matching `*.down.sql`) are the `golang-migrate` migration chain, applied by `make migrate-up`:

- Makefile target `migrate-up` runs `bash scripts/run-migrations.sh up`.
- `scripts/run-migrations.sh` invokes the `migrate` binary against `migrations/postgresql/`.
- `golang-migrate` only recognizes files matching `{version}_{title}.{up|down}.sql`. The numbered pairs match; `0001_primary_schema.sql` does **not**

(no `.up.sql/.down.sql` suffix), so the migrate runner ignores it — by design (see §1b).

The chain now creates the **full, constraint-complete schema**: all 15 reference tables plus `scheduling_queue`, every CHECK constraint, all indexes, the `helix_`-prefixed trigger/function set (migration 015), the `audit_log` DEFAULT partition + partitions through 2027-06, `network_policies`, and `cluster_config` with its seed rows.

1b. `0001_primary_schema.sql` is now a GENERATED, derived artifact

`0001_primary_schema.sql` is no longer a separate hand-maintained schema. It is **generated** as the in-order concatenation of the chain bodies by `migrations/postgresql/.gen_schema.py`, and carries a `GENERATED ARTIFACT – DO NOT EDIT BY HAND` header. It exists solely so the Go applier can stream one file:

- `internal/schema/schema.go SchemaPath()` resolves `migrations/postgresql/0001_primary_schema.sql`.
- `ApplyPrimarySchema()` streams that file to `psql -f -` over stdin, for both local-DSN and `podman exec` (container) transports.

Because the file is *derived from* the chain rather than maintained alongside it, the two cannot describe different schemas. To change the schema, edit the chain (or the bodies in `.gen_schema.py`) and run `python3 migrations/postgresql/.gen_schema.py` to regenerate 0001.

1c. Divergence is now guarded mechanically (no silent drift)

A no-live-DB **drift guard** (`internal/schema/drift_guard_test.go`, runs on every `go test ./internal/schema/...`) parses and normalizes BOTH sources and fails the build if they ever disagree on:

- the normalized SQL text (consolidated 0001 must equal `concat(001..015)`),
- the set of tables/partitions created,
- the set of indexes, triggers, trigger-functions, and named CHECK constraints.

It also carries a mutation/bite proof: injecting a divergent table into one source makes the guard go red. So the historical mismatches below can no longer recur unnoticed.

Historical divergences (now eliminated — kept for the audit trail):

Concept	OLD chain (001–015)	OLD <code>0001_primary_schema.sql</code>
Reservations table	<code>resource_allocations</code> (006)	<code>reservations</code>
Health snapshots table	<code>health_scores</code> (008)	<code>health_snapshots</code>
Advisories table	<code>advisories</code> (009)	<code>llm_advisories</code>
Scheduling queue	<code>scheduling_queue</code> (007) present	absent

Concept	OLD chain (001–015)	OLD 0001_primary_schema.sql
network_policies	absent	present
cluster_config	absent	present (seed rows)
CHECK constraints	none	extensive
audit_log partitions	2026-06 ... 2026-12	+ 2027-* + DEFAULT
audit_log severity CHECK	none	enum CHECK
Trigger/function names	update_updated_at_column, audit_trigger	helix_*
build_artifacts.artifact_type	absent	present
NOT NULL DEFAULT on JSONB/ array cols	nullable (DEFAULT only)	NOT NULL DEFAULT

The per-table sections below document the unified schema (identical in the chain and in the generated 0001).

2. Ordered migration list (golang-migrate chain)

Applied in order by make migrate-up. Each *.up.sql has a paired *.down.sql that drops the indexes and table(s) it created.

#	File	Adds
001	001_create_nodes.up.sql	nodes table + 6 indexes + CHECK constraints
002	002_create_gpu_devices.up.sql	gpu_devices table (FK → nodes ON DELETE CASCADE) + 4 indexes + CHECKs
003	003_create_sessions.up.sql	sessions table (FK → nodes) + 6 indexes + CHECKs
004	004_create_session_windows.up.sql	session_windows table (FK → sessions CASCADE) + 2 indexes + CHECK
005	005_create_session_panes.up.sql	session_panes table (FKs → session_windows CASCADE, nodes, gpu_devices) + 4 indexes + CHECKs
006	006_create_reservations.up.sql	reservations table (FKs → sessions CASCADE, nodes) + 4 indexes + CHECKs
007	007_create_scheduling_queue.up.sql	scheduling_queue table + 3 indexes + CHECKs
008	008_create_health_snapshots.up.sql	

#	File	Adds
		health_snapshots table (FK → nodes CASCADE) + 3 indexes + score-range CHECKs
009	009_create_llm_advisories.up.sql	llm_advisories table + 4 indexes + CHECKs
010	010_create_audit_log.up.sql	audit_log partitioned table + DEFAULT partition + monthly partitions (2026-06...2027-06) + 6 indexes + severity CHECK
011	011_create_build_jobs.up.sql	build_jobs table (FK → nodes) + 5 indexes + CHECKs
012	012_create_build_artifacts.up.sql	build_artifacts table (FK → build_jobs CASCADE) + 4 indexes + CHECKs
013	013_create_users.up.sql	users table (UNIQUE spiffe_id) + 4 indexes + CHECKs
014	014_create_migration_history.up.sql	migration_history table (FKs → sessions, nodes×2) + 4 indexes + CHECKs
015	015_triggers_and_functions.up.sql	network_policies + cluster_config (seeded) tables; helix_update_updated_at_column() + helix_audit_trigger() functions; updated_at triggers on 10 tables; audit triggers on nodes/sessions/build_jobs/users

The generated `0001_primary_schema.sql` is exactly the in-order concatenation of these 15 up-migrations (see §1b). The drift guard (`internal/schema/drift_guard_test.go`) enforces that equivalence on every test run.

3. How migrations are applied

```
# Apply all pending migrations (the 001..015 chain)
make migrate-up
    # → scripts/run-migrations.sh up

# Roll back exactly one migration
make migrate-down
    # → scripts/run-migrations.sh down 1
```

Details (Makefile:76–84, `scripts/run-migrations.sh`):

- `DATABASE_URL` defaults to `postgres://helix:helix@localhost:5432/helix_cluster?sslmode=disable` (Makefile:76); override it for non-local targets. The script can also assemble the DSN from `DB_HOST/DB_PORT/DB_NAME/DB_USER/DB_PASS/DB_SSLMODE` (`scripts/run-migrations.sh`:13–35).

- The script requires the `golang-migrate migrate` binary on `PATH` or at `~/go/bin/migrate`; otherwise it errors with install instructions (`scripts/run-migrations.sh:20-30`).
- Additional sub-commands: `up [N]`, `down [N]`, `version`, `force V`, `create NAME`, `validate` (`up` → `down 15` → `up`), `status` (`scripts/run-migrations.sh:37-123`).

The consolidated `0001_primary_schema.sql` is applied separately by the Go helper `internal/schema.ApplyPrimarySchema`, which pipes the file to `psql -f -` (works for both local DSN and `podman exec` container transports) (`internal/schema/schema.go:279-308`).

4. Trigger functions

Defined in `migrations/postgresql/015_triggers_and_functions.up.sql` (and, identically, in the generated `0001_primary_schema.sql`). Both use the `helix_`-prefixed names.

`helix_update_updated_at_column()` / `update_updated_at_column()`

BEFORE UPDATE row trigger. Sets `NEW.updated_at = NOW()` and returns `NEW`. Attached to every table carrying an `updated_at` column.

`helix_audit_trigger()` / `audit_trigger()`

AFTER INSERT OR UPDATE OR DELETE row trigger, SECURITY DEFINER (per code comment, so it can always INSERT into `audit_log`). It inserts a row into `audit_log` with: - `event_type = <TG_OP>_<TG_TABLE_NAME>` (e.g. `INSERT_nodes`) - `severity = WARNING` for DELETE, else `INFO` - `actor = current_setting('app.current_user', TRUE)` or `'system'` - `resource_type = table name`, `resource_id = row id::TEXT` - `action = the operation`, `details = to_jsonb(OLD|NEW)`

Trigger attachments:

Table	updated_at trigger	audit trigger
nodes	yes (<code>helix_nodes_updated_at</code>)	yes (<code>helix_nodes_audit</code>)
gpu_devices	yes	no
sessions	yes	yes (chain 015 only attaches audit to nodes/ sessions/build_jobs/ users)
reservations	yes	no
users	yes	yes
llm_advisories	yes	no
build_jobs	yes	yes

Table	updated_at trigger	audit trigger
network_policies	yes	no
cluster_config	yes	no

These triggers are identical in the migrate chain and the generated 0001 (migration 015 defines the helix_-prefixed functions and attaches the updated_at triggers to all 10 tables that carry an updated_at column, plus audit triggers to nodes, sessions, build_jobs, users).

5. Tables

Column tables below are grounded in 0001_primary_schema.sql. PK = primary key, FK = foreign key, U = UNIQUE, NN = NOT NULL.

5.1 nodes — cluster node registry

migrations/postgresql/001_create_nodes.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
hostname	VARCHAR(255)	NN
ip_addresses	INET[]	NN
wg_pubkey	TEXT	NN, U (nodes_wg_pubkey_unique)
spiffe_id	TEXT	NN, U (nodes_spiffe_id_unique)
status	VARCHAR(20)	NN, DEFAULT 'JOINING', CHECK ∈ {JOINING, READY, DRAINING, OFFLINE, MAINTENANCE, EVICTED}
role	VARCHAR(20)	NN, DEFAULT 'WORKER', CHECK ∈ {WORKER, CONTROL_PLANE, GPU_WORKER, EDGE, OBSERVER}
cpu_arch	VARCHAR(20)	NN
cpu_cores	INT	NN, CHECK > 0
cpu_threads	INT	NN, CHECK >= cpu_cores
memory_bytes	BIGINT	NN, CHECK > 0
gpu_count	INT	NN, DEFAULT 0, CHECK >= 0
storage_bytes	BIGINT	NN, CHECK > 0

Column	Type	Constraints / Default
labels	JSONB	NN, DEFAULT '{}'
region	VARCHAR(100)	nullable
version	VARCHAR(50)	NN
joined_at	TIMESTAMPTZ	NN, DEFAULT NOW()
last_seen	TIMESTAMPTZ	NN, DEFAULT NOW()
left_at	TIMESTAMPTZ	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_nodes_status, idx_nodes_role, idx_nodes_region, idx_nodes_labels (GIN on labels), idx_nodes_last_seen, idx_nodes_hostname. **Triggers:** helix_nodes_updated_at, helix_nodes_audit.

5.2 gpu_devices — per-node GPU inventory

migrations/postgresql/002_create_gpu_devices.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
node_id	UUID	NN, FK → nodes(id) ON DELETE CASCADE
vendor	VARCHAR(20)	NN, CHECK ∈ {NVIDIA, AMD, INTEL, APPLE, QUALCOMM, OTHER}
model	VARCHAR(100)	NN
driver_version	VARCHAR(50)	NN
api	VARCHAR(20)	NN, CHECK ∈ {CUDA, ROCm, Metal, Vulkan, OpenCL, OTHER}
api_version	VARCHAR(20)	NN
total_memory	BIGINT	NN, CHECK > 0
compute_units	INT	NN, CHECK > 0
features	TEXT[]	NN, DEFAULT '{}'
attributes	JSONB	NN, DEFAULT '{}'
status	VARCHAR(20)	NN, DEFAULT 'AVAILABLE', CHECK ∈ {AVAILABLE, IN_USE, RESERVED, OFFLINE, FAULT}
allocated_to	UUID	nullable (no FK declared)

Column	Type	Constraints / Default
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_gpu_devices_node_id, idx_gpu_devices_status, idx_gpu_devices_vendor, idx_gpu_devices_model. **Trigger:** helix_gpu_devices_updated_at.

5.3 sessions — interactive/batch compute sessions

migrations/postgresql/003_create_sessions.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
name	VARCHAR(255)	NN
owner	TEXT	NN
status	VARCHAR(20)	NN, DEFAULT 'CREATING', CHECK ∈ {CREATING, RUNNING, PAUSED, TERMINATING, TERMINATED, FAILED}
mode	VARCHAR(20)	NN, DEFAULT 'INTERACTIVE', CHECK ∈ {INTERACTIVE, BATCH, DAEMON, NOTEBOOK}
backend	VARCHAR(20)	NN, DEFAULT 'TMUX', CHECK ∈ {TMUX, WASM, CONTAINER, NATIVE}
backend_id	TEXT	nullable
node_id	UUID	FK → nodes (id) , nullable
cpu_request	INT	NN, DEFAULT 1000, CHECK > 0
memory_request	BIGINT	NN, DEFAULT 1073741824, CHECK > 0
gpu_request	JSONB	DEFAULT NULL
priority	INT	NN, DEFAULT 50, CHECK 0..100
labels	JSONB	NN, DEFAULT '{}'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Column	Type	Constraints / Default
started_at	TIMESTAMPTZ	nullable
terminated_at	TIMESTAMPTZ	nullable
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_sessions_owner, idx_sessions_status, idx_sessions_node_id, idx_sessions_mode, idx_sessions_priority, idx_sessions_labels (GIN). **Trigger:** helix_sessions_updated_at (+ helix_sessions_audit in §1b source / chain).

5.4 session_windows — tmux-style windows within a session

migrations/postgresql/004_create_session_windows.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
session_id	UUID	NN, FK → sessions(id) ON DELETE CASCADE
name	VARCHAR(255)	NN
layout	VARCHAR(50)	NN, DEFAULT 'tiled', CHECK ∈ {tiled, even-horizontal, even-vertical, main-horizontal, main-vertical}
active	BOOLEAN	NN, DEFAULT FALSE
crdt_state	JSONB	DEFAULT NULL
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_session_windows_session_id, idx_session_windows_active. No updated_at column / trigger.

5.5 session_panes — panes within a window (placed on a node/GPU)

migrations/postgresql/005_create_session_panes.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
window_id	UUID	NN, FK → session_windows(id) ON DELETE CASCADE

Column	Type	Constraints / Default
node_id	UUID	FK → nodes(id) , nullable
command	TEXT	nullable
working_dir	TEXT	nullable
environment	JSONB	NN, DEFAULT '{}'
cpu_limit	INT	nullable, CHECK IS NULL OR > 0
memory_limit	BIGINT	nullable, CHECK IS NULL OR > 0
gpu_id	UUID	FK → gpu_devices(id) , nullable
status	VARCHAR(20)	NN, DEFAULT 'CREATING', CHECK ∈ {CREATING, RUNNING, STOPPED, FAILED, MIGRATING}
crdt_state	JSONB	DEFAULT NULL
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_session_panes_window_id, idx_session_panes_node_id, idx_session_panes_gpu_id, idx_session_panes_status.

5.6 reservations — resource reservations

migrations/postgresql/006_create_reservations.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
session_id	UUID	NN, FK → sessions(id) ON DELETE CASCADE
node_id	UUID	NN, FK → nodes(id)
cpu_millicores	INT	NN, CHECK > 0
memory_bytes	BIGINT	NN, CHECK > 0
gpu_ids	UUID[]	NN, DEFAULT '{}'
status	VARCHAR(20)	NN, DEFAULT 'PENDING', CHECK ∈ {PENDING, ACTIVE, RELEASED, EXPIRED, FAILED}
expires_at	TIMESTAMPTZ	NN, CHECK > created_at
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Column	Type	Constraints / Default
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_reservations_session_id, idx_reservations_node_id, idx_reservations_status, idx_reservations_expires_at. **Trigger:** helix_reservations_updated_at.

5.7 scheduling_queue — pending scheduling requests

migrations/postgresql/007_create_scheduling_queue.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
request	JSONB	NN
priority	INT	NN, DEFAULT 50
status	VARCHAR(20)	NN, DEFAULT 'PENDING'
scheduled_at	TIMESTAMPTZ	nullable
started_at	TIMESTAMPTZ	nullable
completed_at	TIMESTAMPTZ	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_scheduling_queue_status, idx_scheduling_queue_priority, idx_scheduling_queue_created. **CHECKs:** scheduling_status_valid (status enum), scheduling_priority_range (0..100). **Trigger (015):** helix_scheduling_queue_updated_at.

5.8 health_snapshots — periodic per-node health scores

migrations/postgresql/008_create_health_snapshots.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	BIGSERIAL	PK
node_id	UUID	NN, FK → nodes(id) ON DELETE CASCADE
overall_score	INT	NN, CHECK 0..100
cpu_score	INT	NN, CHECK 0..100
memory_score	INT	NN, CHECK 0..100
disk_score	INT	NN, CHECK 0..100
network_score	INT	NN, CHECK 0..100
gpu_score	INT	NN, CHECK 0..100
temperature_score	INT	NN, CHECK 0..100
services_score	INT	NN, CHECK 0..100

Column	Type	Constraints / Default
predictions	JSONB	NN, DEFAULT '[]'
metrics	JSONB	NN, DEFAULT '{}'
recorded_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_health_snapshots_node_id,
idx_health_snapshots_recorded_at, idx_health_snapshots_overall.

5.9 llm_advisories — LLM “brain” advisories

migrations/postgresql/009_create_llm_advisories.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid() NN, CHECK ∈ {SCALE_UP, SCALE_DOWN, MIGRATE, REBALANCE, DRAIN, EVICT, ALERT, OPTIMIZE, SCHEDULE}
type	VARCHAR(30)	
description	TEXT	NN
rationale	TEXT	NN
proposed_action	JSONB	NN
confidence	FLOAT	NN, CHECK 0.0..1.0
risk_level	VARCHAR(10)	NN, CHECK ∈ {LOW, MEDIUM, HIGH, CRITICAL}
auto_approve	BOOLEAN	NN, DEFAULT FALSE
status	VARCHAR(20)	NN, DEFAULT 'PENDING', CHECK ∈ {PENDING, APPROVED, REJECTED, APPLIED, FAILED, EXPIRED}
applied_by	TEXT	nullable
applied_at	TIMESTAMPTZ	nullable
result	JSONB	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_llm_advisories_status, idx_llm_advisories_type,
idx_llm_advisories_risk_level, idx_llm_advisories_created_at.

Trigger: helix_llm_advisories_updated_at.

5.10 audit_log — append-only audit trail (partitioned by month)

migrations/postgresql/010_create_audit_log.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	BIGSERIAL	part of PK
timestamp	TIMESTAMPTZ	NN, DEFAULT NOW(), partition key
event_type	VARCHAR(50)	NN
severity	VARCHAR(10)	NN, DEFAULT 'INFO', CHECK ∈ {DEBUG, INFO, WARNING, ERROR, CRITICAL}
actor	TEXT	NN
resource_type	VARCHAR(50)	NN
resource_id	TEXT	nullable
action	VARCHAR(50)	NN
details	JSONB	NN, DEFAULT '{}'
source_ip	INET	nullable
session_id	UUID	nullable

Primary key: composite (id, timestamp) (required because the table is PARTITION BY RANGE (timestamp)). **Partitions:** audit_log_default (DEFAULT) + monthly audit_log_2026_06 ... audit_log_2027_06. **Indexes:** idx_audit_log_timestamp, idx_audit_log_event_type, idx_audit_log_actor, idx_audit_log_resource (resource_type, resource_id), idx_audit_log_severity, idx_audit_log_session_id.

5.11 users — shadow of the OIDC/SPIFFE identity provider

migrations/postgresql/013_create_users.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
spiffe_id	TEXT	NN, U (users_spiffe_id_unique)
email	TEXT	nullable
name	VARCHAR(255)	nullable
role	VARCHAR(20)	NN, DEFAULT 'USER', CHECK ∈ {SUPERADMIN, ADMIN, OPERATOR, USER, READONLY, SERVICE}
quota_cpu	INT	NN, DEFAULT 8000, CHECK >= 0

Column	Type	Constraints / Default
quota_memory	BIGINT	NN, DEFAULT 17179869184, CHECK >= 0
quota_gpu	INT	NN, DEFAULT 0, CHECK >= 0
labels	JSONB	NN, DEFAULT '{}'
last_login	TIMESTAMPTZ	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_users_spiffe_id, idx_users_role, idx_users_email, idx_users_labels (GIN). **Triggers:** helix_users_updated_at, helix_users_audit.

5.12 migration_history — session live-migration ledger

migrations/postgresql/014_create_migration_history.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
session_id	UUID	NN, FK → sessions(id)
source_node	UUID	NN, FK → nodes(id)
target_node	UUID	NN, FK → nodes(id)
method	VARCHAR(20)	NN, CHECK ∈ {CHECKPOINT, LIVE, COLD, SNAPSHOT}
duration_ms	INT	NN, CHECK >= 0
data_size_bytes	BIGINT	nullable
success	BOOLEAN	NN
error_message	TEXT	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Additional CHECK: source_node <> target_node (mh_nodes_differ).

Indexes: idx_migration_history_session_id, idx_migration_history_source_node, idx_migration_history_target_node, idx_migration_history_success.

5.13 build_jobs — build subsystem jobs

migrations/postgresql/011_create_build_jobs.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	

Column	Type	Constraints / Default
name	VARCHAR(255)	PK , DEFAULT gen_random_uuid() NN
status	VARCHAR(20)	NN, DEFAULT 'PENDING', CHECK ∈ {PENDING, QUEUED, RUNNING, SUCCEEDED, FAILED, CANCELLED}
mode	VARCHAR(20)	NN, DEFAULT 'BATCH', CHECK ∈ {BATCH, STREAMING, INCREMENTAL, PARALLEL}
cpu_request	INT	NN, DEFAULT 1000, CHECK > 0
memory_request	BIGINT	NN, DEFAULT 1073741824, CHECK > 0
gpu_request	JSONB	DEFAULT NULL
node_id	UUID	FK → nodes(id) , nullable
priority	INT	NN, DEFAULT 50, CHECK 0..100
labels	JSONB	NN, DEFAULT '{}'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
started_at	TIMESTAMPTZ	nullable
completed_at	TIMESTAMPTZ	nullable
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_build_jobs_status, idx_build_jobs_node_id, idx_build_jobs_mode, idx_build_jobs_created_at, idx_build_jobs_priority. **Triggers:** helix_build_jobs_updated_at (+ audit).

5.14 build_artifacts — outputs of build jobs

migrations/postgresql/012_create_build_artifacts.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
job_id	UUID	NN, FK → build_jobs(id) ON DELETE CASCADE

Column	Type	Constraints / Default
artifact_hash	TEXT	NN
size_bytes	BIGINT	NN, CHECK > 0
storage_path	TEXT	NN
artifact_type	VARCHAR(30)	NN, DEFAULT 'BINARY', CHECK ∈ {BINARY, CONTAINER_IMAGE, WASM, LIBRARY, CONFIG, REPORT}
metadata	JSONB	NN, DEFAULT '{}'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_build_artifacts_job_id, idx_build_artifacts_hash, idx_build_artifacts_created_at, idx_build_artifacts_type.

5.15 network_policies — firewall/network policy rules

migrations/postgresql/015_triggers_and_functions.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
name	VARCHAR(255)	NN, U (np_name_unique)
direction	VARCHAR(10)	NN, CHECK ∈ {INGRESS, EGRESS, BOTH}
action	VARCHAR(10)	NN, CHECK ∈ {ALLOW, DENY, LOG, RATE_LIMIT}
protocol	VARCHAR(10)	nullable, CHECK IS NULL OR ∈ {TCP, UDP, ICMP, ESP, AH, ANY}
src_cidr	CIDR	nullable
dst_cidr	CIDR	nullable
src_port_min	INT	nullable, CHECK IS NULL OR 0..65535
src_port_max	INT	nullable
dst_port_min	INT	nullable, CHECK IS NULL OR 0..65535
dst_port_max	INT	nullable
priority	INT	NN, DEFAULT 100, CHECK 0..65535
enabled	BOOLEAN	NN, DEFAULT TRUE

Column	Type	Constraints / Default
labels	JSONB	NN, DEFAULT '{}'
description	TEXT	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Additional CHECKs: src_port_min <= src_port_max (np_src_port_order), dst_port_min <= dst_port_max (np_dst_port_order). **Indexes:** idx_network_policies_direction, idx_network_policies_action, idx_network_policies_enabled, idx_network_policies_priority. **Trigger:** helix_network_policies_updated_at.

5.16 cluster_config — key/value cluster configuration

migrations/postgresql/015_triggers_and_functions.up.sql (= generated 0001)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
key	VARCHAR(255)	NN, U (cc_key_unique)
value	JSONB	NN
description	TEXT	nullable
category	VARCHAR(50)	NN, DEFAULT 'general', CHECK ∈ {general, scheduler, network, security, storage, monitoring, ilm, build}
readonly	BOOLEAN	NN, DEFAULT FALSE
version	INT	NN, DEFAULT 1, CHECK > 0
updated_by	TEXT	NN, DEFAULT 'system'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_cluster_config_key, idx_cluster_config_category. **Trigger:** helix_cluster_config_updated_at. **Seed rows** (INSERT ... ON CONFLICT (key) DO NOTHING, migration 015): scheduler.max_queue_depth=1000, scheduler.default_priority=50, network.mtu=1420, security.session_ttl_hours=8, monitoring.health_interval_s=30.

6. Entity-Relationship diagram

Foreign-key relationships from 0001_primary_schema.sql. network_policies, cluster_config, and audit_log carry no FK references (audit_log.session_id and gpu_devices.allocated_to are plain UUIDs with **no** declared FK).

erDiagram

```
nodes ||--o{ gpu_devices : "node_id (CASCADE)"
nodes ||--o{ sessions : "node_id"
nodes ||--o{ session_panes : "node_id"
nodes ||--o{ reservations : "node_id"
nodes ||--o{ health_snapshots : "node_id (CASCADE)"
nodes ||--o{ build_jobs : "node_id"
nodes ||--o{ migration_history : "source_node"
nodes ||--o{ migration_history : "target_node"

sessions ||--o{ session_windows : "session_id (CASCADE)"
sessions ||--o{ reservations : "session_id (CASCADE)"
sessions ||--o{ migration_history : "session_id"

session_windows ||--o{ session_panes : "window_id (CASCADE)"
gpu_devices ||--o{ session_panes : "gpu_id"

build_jobs ||--o{ build_artifacts : "job_id (CASCADE)"

nodes {
    uuid id PK
    text wg_pubkey UK
    text spiffe_id UK
    varchar status
    varchar role
}
gpu_devices {
    uuid id PK
    uuid node_id FK
    varchar vendor
    varchar status
}
sessions {
    uuid id PK
    uuid node_id FK
    varchar status
    varchar mode
    varchar backend
}
session_windows {
    uuid id PK
    uuid session_id FK
    varchar layout
}
session_panes {
    uuid id PK
```

```

        uuid window_id FK
        uuid node_id FK
        uuid gpu_id FK
        varchar status
    }
    reservations {
        uuid id PK
        uuid session_id FK
        uuid node_id FK
        varchar status
    }
    health_snapshots {
        bigserial id PK
        uuid node_id FK
        int overall_score
    }
    migration_history {
        uuid id PK
        uuid session_id FK
        uuid source_node FK
        uuid target_node FK
        boolean success
    }
    build_jobs {
        uuid id PK
        uuid node_id FK
        varchar status
    }
    build_artifacts {
        uuid id PK
        uuid job_id FK
        text artifact_hash
    }
    users {
        uuid id PK
        text spiffe_id UK
        varchar role
    }
    llm_advisories {
        uuid id PK
        varchar type
        varchar status
    }
    scheduling_queue {
        uuid id PK
        jsonb request
        varchar status
    }
    network_policies {
        uuid id PK
        varchar name UK
        varchar direction
        varchar action
    }
}

```

```

cluster_config {
    uuid id PK
    varchar key UK
    varchar category
}
audit_log {
    bigserial id PK
    timestampz timestamp PK
    varchar event_type
    varchar severity
}

```

Standalone tables (no FK edges): users, llm_advisories,
scheduling_queue, network_policies, cluster_config, audit_log.

7. Source files

- migrations/postgresql/001_*.up.sql ... 015_*.up.sql (+ paired *.down.sql) — the golang-migrate chain applied by make migrate-up. **This is the single canonical schema source** (HXC-1639).
- migrations/postgresql/0001_primary_schema.sql — GENERATED artifact: the in-order concatenation of the chain bodies, applied via internal/schema.ApplyPrimarySchema (psql -f -). Do not hand-edit; regenerate with .gen_schema.py.
- migrations/postgresql/.gen_schema.py — the single-source generator that emits both the chain up/down files and 0001_primary_schema.sql.
- internal/schema/drift_guard_test.go — no-live-DB guard asserting 0001 == concat(001..015) (tables, indexes, triggers, functions, CHECKs); fails the build on any drift.
- scripts/run-migrations.sh — the golang-migrate runner.
- Makefile (migrate-up / migrate-down) — operator entry points.
- internal/schema/schema.go — Go loader/applier (ReadSQL, ReadChainSQL, ApplyPrimarySchema).